ikerlan

MEMBER OF BASQUE RESEARCH & TECHNOLOGY ALLIANCE



User Guide – SDK Fw.

Status of the procedure:

Author	Reviewed by	Approved by
Hebert Luyo	Ane Mijangos	Ane Mijangos
01/04/2026		

INDEX

1. Ikerlan 3
2. SDK Overview 4
2.1. Software Architecture overview 4
2.2. Supported products 4
3. SDK Layers 5
3.1. Application Layer 5
3.2. Middleware Layer 6
3.3. Services Layer 8
3.4. Portability Layer 8
3.5. Client Platform Libraries 8
4. SDK Package description 9
5. SDK Public API 9
5.1. API Usage Model 9
5.2. Status Codes 9
5.3. Initialize Function 10
5.4. Device Mode Functions 10
5.4.1. SGR_SDK_Authenticate_Via_GW 10
5.4.2. SGR_SDK_Verification_FW 11
5.4.3. SGR_SDK_Installation_FW 11
5.5. Gateway Mode Functions 12
5.5.1. Token-to-Server Authentication 12
5.5.2. Server-to-Token Authentication 13
5.5.3. SGR_SDK_GW_Run 13
6. Configuration 14
6.1. SDK Mode 14
6.2. Gateway Token Configuration 15
6.3. Enabling Debug Output 15
6.4. Retry Configuration 16
6.5. CRC Validation 17
6.6. Bitflips Thresholds 17
7. Proof of Concept (PoC) 18
7.1. PoC Scope and Purpose 18
7.2. Architecture 18

7.3. PoC Dependencies 19
7.3.1. GCC and Build Tools 19
7.3.2. MQTT Dependencies (Mosquitto) 19
Project Structure 20
7.4 20
7.4.1. Gateway Project: 20
7.4.2. Device Project (One per Device): 21
7.5. Configuration 21
7.5.1. Gateway (poc_config.h) 22
7.5.2. Each Device (poc_config.h) 22
7.6. Target Environment 22
7.7. Building the PoC 23
7.8. Running the PoC 23
7.8.1. Option 1 – Normal Flow 25
7.8.2. Option 2 — Fault Injection 26
7.8.3. Option 3 — Fault Injection: Error Management Stress 28
7.8.4. Option 4 – FW Verification (Correct RW) 30
7.8.5. Option 5 — FW Verification (Wrong RW) 31
7.8.6. Option 6 – Installation 33
7.9. Notes and Limitations 34

1. IKERLAN

IKERLAN is a leading knowledge transfer technological centre providing competitive value to companies. We seek for excellence in R&D&i, thanks to the continuous adaptation to the needs of our customers and the proximity with the business reality. Faithful to our mission, we have been working daily since 1974 to develop solutions that allow our customers to become more and more competitive. We are a cooperative member of the MONDRAGON Corporation and the Basque Research and Technology Alliance (BRTA).

Thanks to a unique cooperation model, which combines technology transfer activities, internal research and training of highly qualified personnel, IKERLAN is currently the trusted technological partner of major companies in the country. To meet our goal, we are structured in two technological specialisation units:

- ELECTRONICS, INFORMATION AND COMMUNICATION TECHNOLOGIES
- ENERGY AND MECHATRONICS

IKERLAN is a centre that is dynamic and open to the world. We are an accredited agent of the Basque Network of Science, Technology and Innovation and of the Spanish Federation of Technology Centres. We have a major cooperation network integrated by renowned European centres and universities, with which we conduct activities of research and training of researchers.

2. SDK OVERVIEW

2.1. Software Architecture overview

The SDK is designed following a **layered architecture** approach. This design separates responsibilities, protects the public API, and ensures full portability across platforms such as PC and embedded microcontrollers (MCUs).

Each layer has a **clear and well-defined role**, allowing the SDK to evolve internally without impacting user applications.

The SDK is divided into the following layers: Application, Middleware, Services, Utils and Portability.

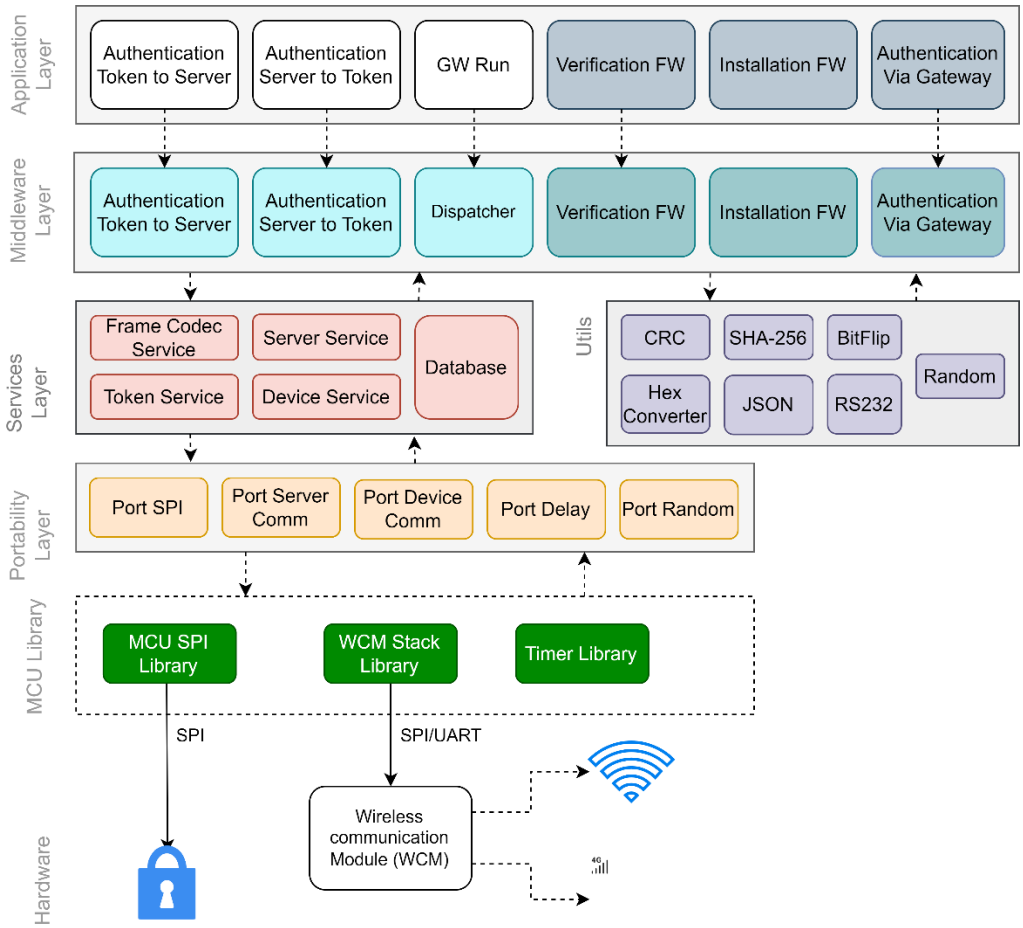


Figure 1. SDK Layered Architecture.

2.2. Supported products

The SDK runs on the following set-up:

Set-up	Port_SPI	Port_COMM
virtualized environment	USB Sandgrain	PC MQTT
MCU 1		
MCU 2		

3. SDK LAYERS

3.1. Application Layer

The Application Layer is the highest-level layer of the SDK and represents the entry point for the client application. It exposes the public SDK API and is responsible for controlling the execution of the Middleware state machine until an operation reaches a final state.

The Middleware layer operates as a state machine — each call to a Middleware function advances the operation by one step and returns either a final status or SGR_MW_BUSY if the operation is still in progress. The Application Layer handles this by iterating the Middleware call in a loop until a conclusive result is obtained, then translating that result into the corresponding sgr_sdk_status_t code returned to the client.

This design gives the client full flexibility over how the function is consumed. As shipped, each API function runs as a blocking call that only returns when the operation completes. However, since the loop is entirely contained within this layer, a client with specific concurrency requirements can replace or wrap this behavior — for example, driving the iteration from a task scheduler or an event loop — without modifying any lower layer.

The API functions exposed by this layer differ depending on the compilation mode:

DEVICE_MODE:

- SGR_SDK_Initialize: Initializes the SDK.
- SGR_SDK_Authenticate_Via_GW: Runs the full authentication flow through the Gateway.
- SGR_SDK_Verification_FW: Performs firmware verification using the crypto chip and reports the result to the Gateway.
- SGR_SDK_Installation: Requests installation authorization from the Gateway. Requires a successful firmware verification in the same session before being called.

GATEWAY_MODE:

- SGR_SDK_Initialize: Initializes the SDK.
- SGR_SDK_Authentication_Token_To_Server: Authenticates the token against the server.
- SGR_SDK_Authentication_Server_To_Token: Authenticates the server against the token.
- SGR_SDK_GW_Run: Unified dispatcher that continuously polls all device channels and automatically routes each incoming request to the correct handler (device authentication, firmware verification, or installation). This is the primary function used by the Gateway application loop.

The full specification of each function — parameters, return values, and usage notes — is covered in the SDK Public API section.

3.2. Middleware Layer

The Middleware Layer implements the authentication protocol logic and acts as the execution engine of the SDK. It is fully platform-independent and contains no hardware-specific code.

This layer is implemented as an explicit state machine. Each call to a Middleware function advances the operation by one step and returns either a conclusive status (SGR_MW_OK, SGR_MW_AUTH_FAILED, SGR_MW_COMM_ERROR, SGR_MW_ERROR) or SGR_MW_BUSY if the operation is still in progress. The Application Layer is responsible for driving this iteration until a final result is obtained.

The Middleware Layer exposes different functions depending on the compilation mode.

DEVICE_MODE:

- **sgr_mw_init:** initializes the middleware and reads the Token ID from the crypto chip
- **sgr_mw_authenticate_via_gw:** drives the full Device↔Gateway authentication flow

GATEWAY_MODE:

- **sgr_mw_init:** initializes the middleware and reads the Token ID from the crypto chip
- **sgr_mw_authentication_token_to_server:** drives Phase 1 of the Gateway's own authentication against the Server (Token→Server)
- **sgr_mw_authentication_server_to_token:** drives Phase 2 of the Gateway's own authentication against the Server (Server→Token)
- **sgr_mw_gw_authenticate_device(int device_idx):** orchestrates the full DEVICE↔GW↔SERVER authentication flow for a specific device channel

Authentication Flow (GATEWAY_MODE)

When the Gateway handles a device authentication request via **sgr_mw_gw_authenticate_device**, the state machine progresses through the following sequence:

1. Receive Auth Request: The Gateway receives the authentication request from the Device, which includes its CRC identifier. The CRC is resolved against the auth_db to retrieve the corresponding Token ID. If the Device is not registered, the flow is rejected immediately.
2. Generate Challenge Word (CW1): The Gateway generates a random 32-byte Challenge Word and stores it in the authentication context.
3. Send CW to Device: CW1 is sent to the Device, which computes a Response Word (RW1) using its internal crypto chip and returns it along with a CW echo (CW2).
4. Verify CW: the Gateway compares CW1 against the received CW2 at bit level. If the difference exceeds CW_BITFLIP_THRESHOLD, authentication fails immediately. If the difference is within the threshold, the flow continues. If there are minor differences, candidate CWs are built for potential use in Error Management.
5. Get RW from Server: The Gateway sends CW1 and the Token ID to the Server, which computes its own Response Word (RW2) using the same secret key.
6. Compare RW: The Gateway compares RW1 (from the Device) against RW2 (from the Server) at bit level. If the difference is within RW_BITFLIP_THRESHOLD, authentication succeeds. If not, the Error Management mechanism is triggered.

7. Error Management: If the RW comparison fails but CW candidates are available, the Gateway systematically tests each candidate CW against the Server to find a matching RW. If a candidate produces a match within the threshold, authentication succeeds. If all candidates are exhausted, authentication fails.
8. Send Result: The result (AUT_OK or AUT_NOK) is sent back to the Device and recorded in the auth_db.
9. Reset: If the operation failed, the state machine resets to the initial state and the Gateway waits for a new authentication request from the Device.

The following diagram illustrates the complete authentication flow:

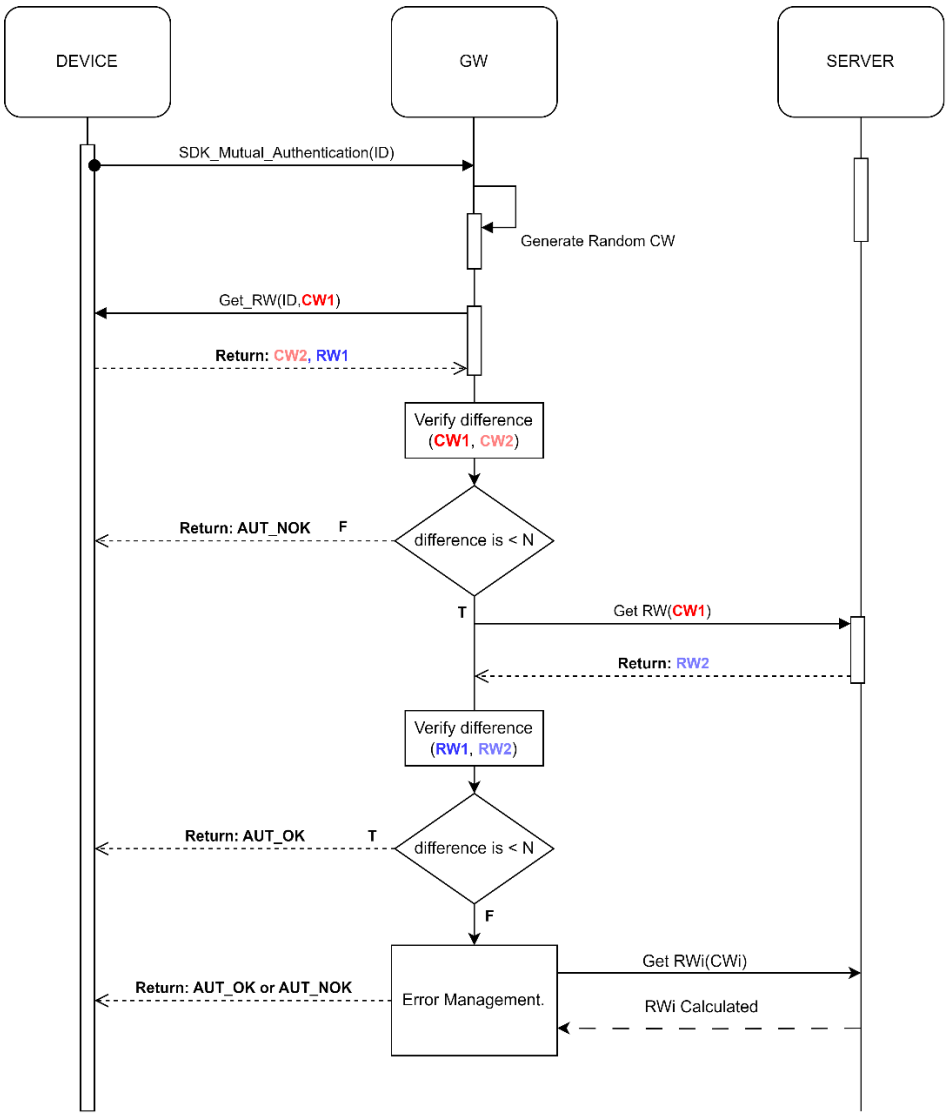


Figure 2. Sequence diagram for authentication with error handling in the GW.

3.3. Services Layer

The Services Layer provides the functional building blocks used by the Middleware Layer to perform concrete actions. Each service is focused on a specific responsibility and can be reused across different authentication flows.

The following services are available in both modes:

- **Frame Codec Service:** handles message framing, encoding, and decoding
- **Token Service:** Manages secure interaction with the crypto chip
- **Server Service:** Abstracts communication with the next node in the authentication chain. In GATEWAY_MODE this service communicates with the Server, and in DEVICE_MODE it communicates with the Gateway. The underlying transport implementation is provided by the Portability Layer and varies depending on the target platform.
- **Utils:** provides utility functions including CRC calculation and hexadecimal conversion.

The following services are available in GATEWAY_MODE only:

- **Device Service:** Handles communication between the Gateway and the connected Devices.
- **Database Service:** Manages the authentication database, including Device registration and CRC-to-TID resolution

3.4. Portability Layer

The **Portability Layer** abstracts platform-dependent capabilities required by the SDK.

This layer provides unified interfaces for:

- SPI communication
- Server communication interfaces
- Timing and delay mechanisms

By isolating platform-specific implementations, the Portability Layer allows the same SDK codebase to run on different platforms without modification.

Porting the SDK to a new platform requires changes only in this layer.

3.5. Client Platform Libraries

The SDK relies on **platform-specific libraries provided by the client** or by the target environment.

These libraries are **not part of the SDK architecture** and are not considered SDK layers.

Typical examples include:

- MCU SPI drivers
- Wireless communication or modem stacks

- Timer or OS abstraction libraries

These libraries are used **exclusively by the Portability Layer** to access platform capabilities. No other SDK layer depends on or interacts with client platform libraries directly.

4. SDK PACKAGE DESCRIPTION

This chapter describes the **package structure of the SGR SDK** and explains how the source code is organized according to the layered architecture.

The SDK package is divided into:

- docs
- examples
- include → Public SDK headers
- port → Port layer interface headers
- src → SDK source code

5. SDK PUBLIC API

This chapter describes the **public application programming interface (API)** exposed by the SGR SDK. The SDK API provides **high-level functions** that allow the client application to perform mutual authentication operations without requiring knowledge of internal protocols, cryptographic mechanisms, or hardware access. All SDK API functions are **synchronous** and maintain internal state within the SDK.

5.1. API Usage Model

The SDK follows a stateful usage model. The available functions and the typical usage flow differ depending on the compilation mode.

DEVICE_MODE:

- Initialize the SDK
- Perform authentication flows

GATEWAY_MODE:

- Initialize the SDK
- Authenticate the Gateway itself against the Server
- Handle incoming Device authentication requests

5.2. Status Codes

All SDK API functions return a value of type sgr_sdk_status_t. The client application must always check the return value to determine the result of the operation.

Value	Description
SGR_SDK_OK	Operation completed successfully
SGR_SDK_ERROR	Generic error
SGR_SDK_INVALID_PARAM	Invalid parameter provided
SGR_SDK_NOT_READY	SDK not ready or not initialized

SGR_SDK_TOKEN_ERROR	Error interacting with the crypto token
SGR_SDK_CRC_ERROR	Data integrity check failed
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_TIMEOUT	Operation timed out
SGR_SDK_INTERNAL_ERROR	Internal SDK error
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_NOT_REGISTERED	Device not registered in the Gateway
SGR_SDK_REGISTRATION_FAILED	Device registration attempt failed
SGR_SDK_NO_DATA	No data available from Device

5.3. Initialize Function

- **Prototype:** *sgr_sdk_status_t SGR_SDK_Initialize(void)*
- **Description**

  Initializes the SGR SDK internal components and prepares the system for authentication operations. This function must be called **once** before invoking any other SDK API.

- **Parameters:** None
- **Return values:** Refer to Section 5.2 for the complete list of status codes.
- **Usage notes**
  - Must be called before any authentication function.
  - In GATEWAY_MODE, this function additionally initializes the authentication database and registers all devices from the whitelist before initializing the middleware.
  - In DEVICE_MODE, only the middleware is initialized.

5.4. Device Mode Functions

5.4.1. SGR_SDK_Authenticate_Via_GW

- **Prototype:** sgr_sdk_status_t SGR_SDK_Authenticate_Via_GW(void);
- **Description:** Performs the full Device authentication flow via the Gateway. The function drives the Middleware state machine until the operation reaches a final state.
- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_ERROR	Internal SDK error

- **Usage notes:**
  - Requires the SDK to be initialized.
  - The function blocks until the authentication flow completes.

5.4.2. SGR_SDK_Verification_FW

- **Prototype:**
  sgr_sdk_status_t SGR_SDK_Verification_FW(const uint8_t *fw_data, uint32_t fw_size, const uint8_t *fw_rw);

- **Description:**
  Performs firmware verification by computing a SHA-256 hash of the provided firmware image and passing it to the crypto chip, which derives a Response Word. The result and the Response Word are sent to the Gateway, which forwards them to the Server for validation. The function drives the Middleware state machine until the operation reaches a final state.

- **Parameters:**

Parameter	Type	Description
fw_data	Const uint8_t *	Pointer to the firmware image buffer.
fw_size	Uint32_t	Size of the firmware image in bytes.
fw_rw	Const uint8_t *	Pointer to a 16-byte buffer where the computed Response Word will be stored. This value must be preserved by the caller for use in a subsequent SGR_SDK_Installation call.

- **Return values:**

Value	Description
SGR_SDK_OK	Firmware verification accepted by the Server
SGR_SDK_AUTH_FAILED	Firmware verification rejected (FW_NOK)
SGR_SDK_INVALID_PARAM	NULL pointer or zero size provided
SGR_SDK_TOKEN_ERROR	Error interacting with the crypto chip
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_ERROR	Internal SDK error

- **Usage notes:**
  - Requires the SDK to be initialized.
  - The function blocks until the operation completes.
  - The Response Word stored in fw_rw upon a successful return (SGR_SDK_OK) is required as a prerequisite for calling SGR_SDK_Installation. It must not be discarded between the two calls.

5.4.3. SGR_SDK_Installation_FW

- **Prototype:** sgr_sdk_status_t SGR_SDK_Installation(void)

- **Description:** Requests installation authorization from the Gateway. The SDK sends the firmware Response Word obtained during a previous successful firmware

verification along with the device credentials. The Gateway forwards the request to the Server, which authorizes or denies the installation.

- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Installation authorized
SGR_SDK_AUTH_FAILED	Installation denied by the Server
SGR_SDK_TOKEN_ERROR	Error interacting with the crypto chip
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_ERROR	Internal SDK error

- **Usage notes:**
  - Requires the SDK to be initialized.
  - SGR_SDK_Verification_FW must have completed successfully in the same session before calling this function. The SDK retains the firmware Response Word internally after a successful verification.
  - The function blocks until the operation completes.

5.5. Gateway Mode Functions

5.5.1. Token-to-Server Authentication

- **Description**

  Performs **token-to-server authentication**. This function executes the complete authentication flow, including:

  - Requesting a Challenge Word (CW) from the server
  - Computing the Response Word (RW) using the secure token
  - Sending the Response Word back to the server
  - Processing the authentication result

  All steps are handled internally by the SDK.

- **Prototype:**
  sgr_sdk_status_t SGR_SDK_Authentication_Token_To_Server(void)

- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_INTERNAL_ERROR	Internal SDK error

- **Usage notes**
  - Requires the SDK to be initialized.
  - The function blocks until the authentication flow completes.
  - Retry mechanisms are handled internally by the SDK.

5.5.2. Server-to-Token Authentication

- **Description**

  Performs **server-to-token authentication**.

  This function executes the reverse authentication flow, including:

  - Generating a Challenge Word from the token
  - Requesting and validating the Response Word
  - Reporting the authentication result

- **Prototype**

  sgr_sdk_status_t SGR_SDK_Authentication_Server_To_Token(void)

- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_INTERNAL_ERROR	Internal SDK error

- **Usage notes**
  - Requires the SDK to be initialized.
  - Intended for mutual authentication scenarios.
  - Hardware and protocol details are fully abstracted.

5.5.3. SGR_SDK_GW_Run

- **Prototype:** sgr_sdk_status_t SGR_SDK_GW_Run(void)
- **Description:**

Unified Gateway dispatcher. On each call it polls all device channels (up to MAX_DEVICES) and automatically routes each incoming request to the correct handler — device authentication, firmware verification, or installation — based on the frame type received. This function replaces the need to call individual handlers and is the recommended entry point for the Gateway application loop.

- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_BUSY	All channels are idle or still in progress — no flow completed on this call
SGR_SDK_OK	One or more flows completed successfully
SGR_SDK_AUTH_FAILED	A flow completed with authentication failure
SGR_SDK_COMM_ERROR	Critical and unrecoverable communication error
SGR_SDK_ERROR	Critical and unrecoverable internal error

- **Usage notes:**

  - Requires the SDK to be initialized.
  - Intended to be called in a tight loop from the client application. The caller should continue looping on SGR_SDK_BUSY.
    Frame type detection and channel routing are handled internally — the client does not need to inspect incoming frames or decide which handler to invoke.

    SGR_SDK_GW_Handle_Device_Auth, SGR_SDK_GW_Handle_FW_Verification, and SGR_SDK_GW_Handle_Installation remain available for clients that require explicit per-flow control, but SGR_SDK_GW_Run is the recommended interface for standard integrations.

6. CONFIGURATION

The SGR SDK provides a centralized configuration file, sgr_config.h, which controls the internal behavior of the SDK. All parameters take effect at compile time. This file covers SDK mode selection, debug output, retry limits, and CRC validation.

Platform-specific configuration such as network parameters, broker settings, or hardware port assignments is not part of the SDK and must be handled in the application or PoC configuration file (poc_config.h).

6.1. SDK Mode

The SDK_MODE parameter defines the compilation mode and determines which functions and port implementations are included in the build:

```
#define SDK_MODE GATEWAY_MODE /* or DEVICE_MODE */
```

Value	Description
DEVICE_MODE	Token node — authenticates via Gateway
GATEWAY_MODE	Gateway node — handles Device requests and Cloud communication

The SDK will emit a compile-time error if SDK_MODE is set to an invalid value.

6.2. Device Registration (Whitelist)

Before building the Gateway, the TID of each Device must be registered in:

sdk_fw/src/services/database/auth_db_whitelist.c

Add each device TID as a 64-character hex string in the auth_db_whitelist[] array:

```
const char *auth_db_whitelist[] = {
  "8000000000000000000000000000000185dcba84764025caa98c6c43032c8b433",
// Device 1
  "8000000000000000000000000000001813ee114f3a061a3d6929ac91d9a1ca20",
// Device 2
  /* Add more TIDs here */
};
```

If a Device's TID is not present in this list, the Gateway will reject it immediately at the first step of the authentication flow with SGR_SDK_NOT_REGISTERED. The Gateway must be rebuilt after any change to this file.

6.3. Gateway Token Configuration

This parameter is only applicable in GATEWAY_MODE and controls whether the Gateway requires its own crypto token connected via USB/SPI.

```
#define GW_CRYPTO_ENABLED 0   /* 0 = no token required, 1 = token required */
```

When set to 0, the Gateway skips the crypto chip initialization during SGR_SDK_Initialize and does not require a token. This is the default configuration.

When set to 1, the Gateway expects its own token connected via USB/SPI and reads its Token ID during initialization. This enables the Gateway's own Token-to-Server and Server-to-Token authentication flows against the Server.

This parameter has no effect in DEVICE_MODE, where the crypto token is always required.

6.4. Enabling Debug Output

The SDK exposes six independent debug flags, one per module. Each flag controls the log output of its corresponding layer independently.

```
#define MW_DEBUG                1  /* middleware layer logs                */
#define PORT_COMM_DEBUG         1  /* server communication layer logs      */
#define PORT_COMM_INPUT_DEBUG   1  /* input communication layer logs       */
#define PORT_COMM_DEVICE_DEBUG  1  /* device socket communication logs     */
#define APP_GW_DEBUG            1  /* gateway application layer logs       */
#define MW_GW_DEBUG             1  /* gateway middleware logs              */
```

To enable debug logging for a module, set its corresponding flag to 1. To disable it, set it to 0. All flags default to 1.

When a flag is set to 0 or undefined, all log calls for that module are replaced by empty statements at compile time, producing no code and no runtime overhead. This makes it safe to ship production builds with debug flags disabled, as there is no performance penalty.

All flags can be overridden from the build system without modifying the SDK file:

-DMW_DEBUG=0 -DPORT_COMM_DEBUG=0

The log prefix for each module makes it easy to identify the source of each line in the output:

Flag	Log prefix	Scope
MW_DEBUG	[MW]	Middleware state machine steps
PORT_COMM_DEBUG	[PORT][MQTT]	MQTT publish and receive operations
PORT_COMM_INPUT_DEBUG	[PORT][INPUT]	Incoming communication parsing
PORT_COMM_DEVICE_DEBUG	[PORT][DEV]	Device socket communication
APP_GW_DEBUG	[APP][GW]	Gateway application flow
MW_GW_DEBUG	[MW][GW]	Gateway middleware internal steps

Note that the MQTT transport layer is part of the Proof of Concept (PoC) implementation and is not a core component of the SDK itself. In a production integration, the communication layer would be replaced by the target platform's own transport mechanism, and the [PORT][MQTT] logs would reflect that implementation instead.

6.5. Retry Configuration

The SDK exposes two independent retry parameters, one for each side of the authentication chain:

```
#define MAX_RETRIES       4   /* Device-side retry limit   */
#define GW_MAX_RETRIES  3   /* Gateway-side retry limit  */
```

MAX_RETRIES defines how many times the Device SDK will retry a failed operation before reporting an error. Applied to Device-side flows such as authentication via Gateway, firmware verification, and installation.

GW_MAX_RETRIES defines how many times the Gateway SDK will retry a failed operation on its side before reporting an error. Applied to Gateway-side flows such as querying the Server for a Response Word during device authentication and Error Management.

A value of 0 means no retries will be performed and only one attempt will be made. If a negative value is set, the SDK will automatically override it and enforce the minimum of 0:

```
#if MAX_RETRIES < 0
        #warning "MAX_RETRIES is less than 0, forcing to 0"
        #undef MAX_RETRIES
        #define MAX_RETRIES 0
```

```
#endif
```

This validation happens at compile time and will emit a compiler warning to notify the developer that the value was corrected, so the behavior is never silent.

Both parameters can be overridden from the build system without modifying the SDK file:

-DMAX_RETRIES=2 -DGW_MAX_RETRIES=1

Important: The parameter validation block at the bottom of sdk_sgr_config.h must not be modified. It exists to guarantee safe minimum values are always enforced regardless of what is configured above it.

6.6. CRC Validation

The SDK implements CRC16 validation for all received messages to ensure data integrity. This feature can be disabled for testing or debugging purposes.

```
#define CRC_ENABLE 0
```

The default value is **0 (disabled)**. If CRC_ENABLE is not explicitly passed from the build system, CRC validation will be inactive. For production builds it is strongly recommended to enable it:

-DCRC_ENABLE=1

When disabled, all CRC checks in the protocol layer are skipped at compile time, producing no runtime overhead. When enabled, every received message is validated against its CRC before processing, and any mismatch results in SGR_SDK_CRC_ERROR.

6.7. Bitflips Thresholds

The SDK uses two configurable thresholds to control the tolerance for bit-level differences during the authentication flow. These thresholds directly govern when the Error Management mechanism is triggered and when authentication fails outright.

```
#define CW_BITFLIP_THRESHOLD    3   /* max bit diff tolerated in CW comparison */

#define RW_BITFLIP_THRESHOLD    3   /* max bit diff tolerated in RW comparison */
```

- **CW_BITFLIP_THRESHOLD** — applied by the Gateway when comparing the original CW it generated against the CW echo received from the Device. If the bit difference exceeds this value, authentication fails immediately. If it is within the threshold, the flow continues and candidate CWs are built for potential use in Error Management.
- **RW_BITFLIP_THRESHOLD** — applied by the Gateway when comparing the Response Word received from the Device against the one computed by the Server. If the bit difference is within this value, authentication succeeds directly. If it exceeds it, Error Management is triggered.

Both parameters default to 3 and should be tuned based on the noise characteristics of the physical communication channel. A lower value produces stricter authentication; a higher value increases tolerance but also reduces sensitivity to genuine errors.

Both can be overridden from the build system without modifying the SDK file:

-DCW_BITFLIP_THRESHOLD=2 -DRW_BITFLIP_THRESHOLD=2

If a negative value is provided, the SDK will automatically override it and enforce the minimum of 0, emitting a compiler warning to notify the developer.

Important: Setting either threshold to 0 means any single bit difference will trigger Error Management (for RW) or fail immediately (for CW). This is a valid strict configuration but must be set explicitly and intentionally.

7. PROOF OF CONCEPT (POC)

This chapter describes the Proof of Concept provided with the SGR SDK. The PoC is a reference implementation used for validation, testing, and demonstration purposes. It is not a mandatory component of the SDK.

7.1. PoC Scope and Purpose

The PoC validates the following aspects of the SDK:

- Correct initialization of both the Device and Gateway executables
- Mutual authentication flow between Device, Gateway and Server
- Concurrent handling of multiple Device authentication requests by the Gateway
- Fault injection and Error Management mechanisms

7.2. Architecture

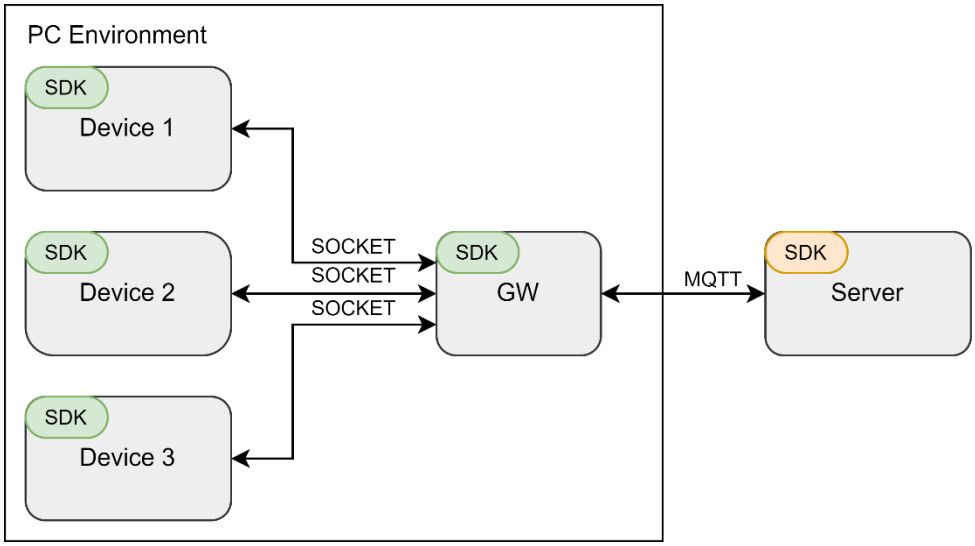


Figure 3. PoC Architecture

The PoC runs entirely in a PC environment and consists of four independent executables:

- **3 Device executables** — each compiled in DEVICE_MODE, each with its own crypto token connected via USB/SPI
- **1 Gateway executable** — compiled in GATEWAY_MODE, connected to all three Devices via TCP socket and to the Server via MQTT

Each Device connects to the Gateway on a dedicated TCP socket port. The Gateway listens on three consecutive ports starting from PORT_BASE:

Executable	Socket Port	Description
Device 1	8080	Connects to GW channel 0
Device 2	8081	Connects to GW channel 1
Device 3	8082	Connects to GW channel 2

7.3. PoC Dependencies

This section lists the tools and libraries required only for building and running the PoC. These dependencies are not required to use the SDK on other platforms (e.g., embedded MCUs), where equivalent platform-specific implementations are expected.

7.3.1. GCC and Build Tools

The PoC is written in C and requires a standard GCC toolchain.

- **Linux:**

sudo apt update

sudo apt install build-essential

- **Windows:**

Install MSYS2 from https://www.msys2.org and follow the official setup instructions. Ensure the MinGW64 toolchain is installed and available in the terminal. All subsequent build and run commands must be executed from the **MSYS2 MinGW64 terminal**.

This installs:

- GCC compiler
- Make build system
- Standard C libraries and headers

7.3.2. MQTT Dependencies (Mosquitto)

The PoC uses MQTT for server communication.

The Mosquitto client development library is required:

- **Linux:**

sudo apt install libmosquitto-dev

- **Windows:**

Install Mosquitto from the official website and ensure the development libraries are available to the MinGW64 build environment. Refer to the Mosquitto documentation for Windows installation details.

This library provides:

MQTT client headers

7.4. Project Structure

The PoC consist of two separate projects, one for the Gateway and one for the Device.

7.4.1. Gateway Project:

PoC_SGR/
- **.vscode/** → VSCode debug configuration
  - launch.json → Debug settings (Linux + Windows)
  - tasks.json → Build tasks
- **app/** → PoC Application
  - **config_poc.h** → Gateway configuration
  - **maing_gw.c** → Gateway main application
  - Makefile → Build system
  - **port/** → Platform-specific implementations
    - port_comm.c → MQTT communication layer
    - port_comm_device.c →Device socket communication
    - port_delay.c →Timing functions
    - port_spi_usb.c → USB/SPI token interface
- **sdk_fw/** → SGR SDK (read-only)
  - **docs** → Documentation about SDK
  - **examples** → Some examples of PORT layer implementation
  - **include/** → Public SDK headers
  - **port/** → Port layer interface headers
  - **src/** → SDK source code
- **setup-debug.sh** → Debug configuration script (Linux only)

Note: The sdk_fw/ directory should not be modified except for src/services/database/auth_db_whitelist.c, which contains the list of authorized device TIDs and must be updated before building the Gateway.

7.4.2. Device Project (One per Device):

PoC_SGR/

- **.vscode/** → VSCode debug configuration
  - launch.json → Debug settings (Linux + Windows)
  - tasks.json → Build tasks
- **app/** → PoC Application
  - **config_poc.h** → Gateway configuration
  - **maing_device.c** → Gateway main application
  - Makefile → Build system
  - fault_injection
    - fi_device.c → Fault injection implementation
    - fi_device.h → Fault injection conriguration
  - **port/** → Platform-specific implementations
    - port_comm.c → MQTT communication layer
    - port_comm_device.c →Device socket communication
    - port_delay.c →Timing functions
    - port_spi_usb.c → USB/SPI token interface
- **sdk_fw/** → SGR SDK (read-only)
  - **docs** → Documentation about SDK
  - **examples** → Some examples of PORT layer implementation
  - **include/** → Public SDK headers
  - **port/** → Port layer interface headers
  - **src/** → SDK source code
- **setup-debug.sh** → Debug configuration script (Linux only)

Key Points:

- The sdk_fw/ directory contains the SDK core and should not be modified
- Configuration is done exclusively through sdk_fw/include/sdk_sgr_config.h
- The app/ directory contains the PoC application and platform-specific code

7.5. Configuration

Before building the PoC, two configuration files must be set up for each executable:

sdk_sgr_config.h defines the SDK compilation mode and all internal parameters. This file must not be modified directly. The SDK_MODE parameter is passed from the Makefile at compile time:

-DSDK_MODE=DEVICE_MODE /* for Device executables */

-DSDK_MODE=GATEWAY_MODE /* for Gateway executable */

This is already handled by the provided Makefiles — no manual changes are required unless you need to override other parameters such as MAX_RETRIES or CRC_ENABLE.

poc_config.h — defines all environment-specific parameters. This is the only file that needs to be adapted to your hardware and network setup. Do not modify any other SDK file.

7.5.1. Gateway (poc_config.h)

```
#define MAX_DEVICES 3 /* number of device channels */
#define PORT_BASE 8080 /* base TCP port — GW listens on 8080, 8081, 8082 */
```

Additionally configure the MQTT broker parameters:

```
#define MQTT_BROKER "172.16.124.248" /* Broker IP or hostname */
#define MQTT_PORT 1883 /* Broker port */
#define MQTT_USER "admin" /* Broker username */
#define MQTT_PASS "changeme" /* Broker password */
#define MQTT_KEEPALIVE 60 /* Keepalive in seconds */
```

Configure the USB port of the crypto token for each Device:

Windows:

```
#define USB_COM_PORT 18 // Change to your COM port number
```

Linux:

```
#define USB_TTY_PORT 0    // Change to your ttyACM number
```

7.5.2. Each Device (poc_config.h)

```
#define PORT 8080 /* socket port matching the GW channel */
#define SERVER_IP "127.0.0.1"
```

Note: SERVER_IP refers to the IP address of the **Gateway**, not the MQTT broker or the cloud server. For a local setup where all executables run on the same machine, use 127.0.0.1.

Windows:

```
#define USB_COM_PORT 18 // Change to your COM port number
```

Linux:

```
#define USB_TTY_PORT 0    // Change to your ttyACM number
```

7.6. Target Environment

The PoC has been developed and tested in the following environment:

- Operating System: Windows (MinGW64 / MSYS2). Linux support is expected but has not been validated.
- Execution mode: 4 independent user-space processes (1 GW + 3 Devices)
- Communication: TCP Socket (Device ↔ GW) and MQTT over TCP/IP (GW ↔ Server)
- Hardware: SGR crypto token connected via USB/SPI (one per Device)
- Build system: Make with MinGW64 toolchain

Porting or executing the PoC on a different environment may require adaptations outside the scope of this document.

7.7. Building the PoC

- Step 1 — Build the Gateway:

  cd poc_sdk_gateway/app

  make

- Step 2 — Build each Device:

  cd poc_sdk_device_1/app

  make

  cd poc_sdk_device_2/app

  make

  cd poc_sdk_device_3/app

  make

If the build completes successfully the following executables will be generated in each app/ directory:

Executable	Description
Poc_sdk_gateway/app/test_gw.exe	Gateway executable
poc_sdk_device_1/app/test_device.exe	Device 1 executable
poc_sdk_device_2/app/test_device.exe	Device 2 executable
poc_sdk_device_3/app/test_device.exe	Device 3 executable

Clean build:

make clean && make

7.8. Running the PoC

Before starting any executable, ensure the MQTT broker is running and reachable at the address configured in poc_config.h. The Gateway connects to the broker on startup — if the broker is not available, initialization will fail.

The Gateway must be started first and must be fully initialized before launching any Device. Open a separate terminal for each executable.

Then start the Gateway:

Terminal 1 – Start the Gateway

```
cd poc_sdk_gateway/app
./test_gw.exe
```

The Gateway is ready then the following output appears:

```
=====================================================
 SGR SDK — GATEWAY
=====================================================
[INIT] Initializing GW SDK...
[AUTH_DB] Initialized (100 slots)
[AUTH_DB] Registered: 80000000...fbf6 (CRC32=0x...)
[AUTH_DB] Registered: ...
[AUTH_DB] Registered: ...
[AUTH_DB] Registered: ...
 [MW] SDK ready (no crypto chip)
 OK — GW SDK initialized
[GW] Listening for device requests...
-----------------------------------------------------
```

Terminal 2, 3, 4 – Start each Device:

```
cd poc_sdk_device_X/app
./test_device.exe
```

Each device presents an interactive menu with six test options:

```
Each device presents an interactive menu with six test options:

=====================================================
 Select test option:
 --- Authentication ---
 1. Normal flow (no fault injection)
 2. Fault injection (CW: 2 bits, CW echo: 1 bit, RW: 1 bit)
 3. Fault injection — Error Management stress (CW: 3 bits, CW echo: 2 bits)
 --- FW Verification ---
 4. FW Verification — correct RW  (expect FW_OK, saves RW_verify)
 5. FW Verification — wrong RW    (expect FW_NOK)
 --- Installation ---
 6. Installation    — request install auth (run option 4 first)
 ---
 0. Exit
=====================================================
 Option:
```

The three Devices can be launched independently and do not need to be started simultaneously. Each Device connects to its assigned GW channel based on the PORT value configured in its poc_config.h.

7.8.1. Option 1 – Normal Flow

This option executes the complete authentication flow without any fault injection. Both the CW and RW are transmitted without corruption, and the Gateway verifies them without needing to invoke the Error Management mechanism.

```
[ AUTH ] Authenticating via Gateway...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — Authenticate via GW
[MW] ──────────────────────────────────────────
[MW] [1/4] Sending auth request to GW...
[MW] [1/4] TID:
800000000000000000000000000001813ee114f3a061a3d6929ac91d9a1ca20
[SERVER] Auth request — CRC32: 0x637AE904
[MW] [1/4] Auth request sent ✓
[MW] [2/4] Challenge Word received from GW ✓
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2

[MW] [3/4] Computing Response Word on crypto chip...
[MW] [3/4] Response Word computed ✓
[MW] [3/4] CW echo:
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2
[MW] [3/4] RW:
dc6c0db144ac810b9ead8aa438121236
[MW] [4/4] GW confirmed device — AUT_OK ✓
  ✓ Authentication OK
```

Gateway Output:

```
[MW] [CH1][1/6] Auth request received — CRC32=0x637AE904 ✓
[MW] [CH1][2/6] Challenge Word generated ✓
[MW] [CH1][3/6] Challenge Word sent to device ✓
[MW] [CH1][4/6] CW echo + RW received from device ✓
[MW] [CH1][DEBUG] RW_device:
dc6c0db144ac810b9ead8aa438121236
[MW] [CH1][DEBUG] CW1 original:
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2
[MW] [CH1][DEBUG] CW2 received:
54ba861c9fb73a3fb91e245ff9bb72470f52697395df9f75e7226a97355b98c2
[MW] [CH1][DEBUG] Differing bit positions:
[MW] [CH1][4/6] CW verified — bit diff=0 ✓
[MW] [CH1][5/6] RW request sent to server ✓
[MW] [CH1][5/6] RW received from server ✓
[MW] [CH1][DEBUG] RW_server:
dc6c0db144ac810b9ead8aa438121236
[MW] [CH1][DEBUG] RW compare — device vs server — bit_diff=0
[MW] [CH1][6/6] RW match — bit diff=0 — AUT_OK ✓
[MW] [CH1][6/6] Auth result sent to device — AUT_OK
```

The Device sends its CRC identifier to the Gateway. The Gateway resolves the CRC against the authentication database, generates a random 32-byte Challenge Word (CW1) and sends it to the Device. The Device passes the CW to the crypto chip, which computes a Response Word (RW). The Device returns the CW echo and RW to the Gateway. The Gateway verifies that CW1 and the received CW echo are identical (bit diff = 0), then queries the Server for its own RW computation. Both RWs match exactly (bit diff = 0) and AUT_OK is returned to the Device.

7.8.2. Option 2 — Fault Injection

This option simulates communication channel errors by injecting controlled bit corruptions into the CW and RW exchanged between the Device and the Gateway. The purpose is to validate the Gateway's bitflip tolerance and Error Management mechanism under realistic fault conditions.

Target	Bits injected	Location
CW received by Device	2 bits	port_comm_receive
CW echo sent by Device	1 additional bit	port_comm_send
RW sent by Device	1 bit	port_comm_send

The corruptions are applied at the Port layer of the Device, simulating errors that could occur in a real communication channel.

Device Output:

```
[ AUTH ] Authenticating via Gateway...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — Authenticate via GW
[MW] ──────────────────────────────────────────
[MW] [1/4] Sending auth request to GW...
[MW] [1/4] TID:
800000000000000000000000000001813ee114f3a061a3d6929ac91d9a1ca20
[SERVER] Auth request — CRC32: 0x637AE904
[MW] [1/4] Auth request sent ✓
[MW] [2/4] Challenge Word received from GW ✓
b2c535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [3/4] Computing Response Word on crypto chip...
[MW] [3/4] Response Word computed ✓
[MW] [3/4] CW echo:
b2c535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [3/4] RW:
9867a76cc1a236c5c1c5fb92edc48063
[MW] [4/4] GW confirmed device — AUT_OK ✓
  ✓ Authentication OK
```

Gateway Output:

```
[MW] [CH1][1/6] Auth request received — CRC32=0x637AE904 ✓
[MW] [CH1][2/6] Challenge Word generated ✓
[MW] [CH1][3/6] Challenge Word sent to device ✓
[MW] [CH1][4/6] CW echo + RW received from device ✓
[MW] [CH1][DEBUG] RW_device:
b867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][DEBUG] CW1 original:
324535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][DEBUG] CW2 received:
bac535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][DEBUG] Differing bit positions:
3 7 15
[MW] [CH1][4/6] CW verified — bit diff=3 — 7 candidates built ✓
[MW] [CH1][5/6] RW request sent to server ✓
[MW] [CH1][5/6] RW received from server ✓
[MW] [CH1][DEBUG] RW_server:
436188c7fa4cbc931c7bbcaba7bdbe8a
[MW] [CH1][DEBUG] RW compare — device vs server — bit_diff=75
[MW] [CH1][6/6] RW mismatch — bit diff=75 > threshold=3
[MW] [CH1][ERROR MGT] Starting Error Management — 7 candidates
[MW] [CH1][ERROR MGT] Testing candidate 1 / 7
[MW] [CH1][ERROR MGT] Candidate 1 / 7
[MW] [CH1][ERROR MGT] CW candidate:
bac535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][ERROR MGT] RW_device:
b867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][ERROR MGT] RW_server:
e1ee646c6f2e84937c815a0b61f8bcf7
[MW] [CH1][ERROR MGT] bit_diff=56 — no match, trying next
[MW] [CH1][ERROR MGT] Candidate 1 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 2 / 7
[MW] [CH1][ERROR MGT] Candidate 2 / 7
[MW] [CH1][ERROR MGT] CW candidate:
b2c535e09f8fe94cfccf5f84c469fca9a425fc57d12f63cf9f62365650c5ead8
[MW] [CH1][ERROR MGT] RW_device:
b867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][ERROR MGT] RW_server:
9867a76cc1a236c5c1c5fb92edc48063
[MW] [CH1][ERROR MGT] bit_diff=1 — MATCH ✓
[MW] [CH1][ERROR MGT] Candidate 2 matched — AUT_OK ✓
[MW] [CH1][6/6] Auth result sent to device — AUT_OK
```

The Device receives CW1 from the Gateway and the Port layer injects 2 bit corruptions, producing a corrupted CW. The crypto chip computes the RW using this corrupted CW. Before sending back to the Gateway, 1 additional bit is injected in the CW echo (total 3 bits different from CW1) and 1 bit is injected in the RW.

The Gateway detects 3 bits of difference between CW1 and the received CW echo (positions 3, 7 and 15) and builds 7 candidates. The initial RW comparison fails with a bit diff of 75, well above the threshold. Error Management is triggered and tests each candidate against the Server. Candidate 1 fails. Candidate 2 matches with bit diff = 1, within the threshold, and AUT_OK is returned.

7.8.3. Option 3 — Fault Injection: Error Management Stress

This option is designed to stress-test the Error Management mechanism by forcing the Gateway to exhaust a larger number of candidates before finding the correct match. The fault injection configuration is more aggressive than option 2, requiring the Gateway to test all 7 candidates before resolving the authentication.

Target	Bits injected	Location
CW received by Device	1 bit	Port_comm_receive
CW echo sent by Device	2 additional bits	Port_comm_send

Device Ouput:

```
[ AUTH ] Authenticating via Gateway...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — Authenticate via GW
[MW] ──────────────────────────────────────────
[MW] [1/4] Sending auth request to GW...
[MW] [1/4] TID:
800000000000000000000000000001813ee114f3a061a3d6929ac91d9a1ca20
[SERVER] Auth request — CRC32: 0x637AE904
[MW] [1/4] Auth request sent ✓
[MW] [2/4] Challenge Word received from GW ✓
7a6ae7b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [3/4] Computing Response Word on crypto chip...
[MW] [3/4] Response Word computed ✓
[MW] [3/4] CW echo:
7a6ae7b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [3/4] RW:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [4/4] GW confirmed device — AUT_OK ✓
  ✓ Authentication OK
```

Gateway Output:

```
[MW] [CH1][1/6] Auth request received — CRC32=0x637AE904 ✓
[MW] [CH1][2/6] Challenge Word generated ✓
[MW] [CH1][3/6] Challenge Word sent to device ✓
[MW] [CH1][4/6] CW echo + RW received from device ✓
[MW] [CH1][DEBUG] RW_device:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [CH1][DEBUG] CW1 original:
fa6ae7b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [CH1][DEBUG] CW2 received:
7aea67b664398dc963d721030b17f600dd8736d4eb0023423d7ba32f7226fc55
[MW] [CH1][DEBUG] Differing bit positions:
7 15 23
[MW] [CH1][4/6] CW verified — bit diff=3 — 7 candidates built ✓
[MW] [CH1][5/6] RW request sent to server ✓
[MW] [CH1][5/6] RW received from server ✓
[MW] [CH1][DEBUG] RW_server:
0e216d55241cc859f2f12b42de7cce8b
[MW] [CH1][DEBUG] RW compare — device vs server — bit_diff=69
[MW] [CH1][6/6] RW mismatch — bit diff=69 > threshold=3
[MW] [CH1][ERROR MGT] Starting Error Management — 7 candidates
[MW] [CH1][ERROR MGT] Testing candidate 1 / 7
[MW] [CH1][ERROR MGT] Candidate 1 / 7 — bit_diff=57 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 1 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 2 / 7
[MW] [CH1][ERROR MGT] Candidate 2 / 7 — bit_diff=63 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 2 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 3 / 7
[MW] [CH1][ERROR MGT] Candidate 3 / 7 — bit_diff=62 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 3 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 4 / 7
[MW] [CH1][ERROR MGT] Candidate 4 / 7 — bit_diff=75 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 4 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 5 / 7
[MW] [CH1][ERROR MGT] Candidate 5 / 7 — bit_diff=67 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 5 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 6 / 7
[MW] [CH1][ERROR MGT] Candidate 6 / 7 — bit_diff=60 — no match,
trying next
[MW] [CH1][ERROR MGT] Candidate 6 failed — trying next
[MW] [CH1][ERROR MGT] Testing candidate 7 / 7
[MW] [CH1][ERROR MGT] Candidate 7 / 7
[MW] [CH1][ERROR MGT] RW_device:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [CH1][ERROR MGT] RW_server:
a1b8fc9acaa3ec465a928abefd4e1a53
[MW] [CH1][ERROR MGT] bit_diff=0 — MATCH ✓
[MW] [CH1][ERROR MGT] Candidate 7 matched — AUT_OK ✓
[MW] [CH1][6/6] Auth result sent to device — AUT_OK
```

Device receives CW1 from the Gateway and the Port layer injects 1 bit corruption. The crypto chip computes the RW using this corrupted CW. Before sending back to the Gateway, 2 additional bits are injected in the CW echo, resulting in a total of 3 bits different from CW1.

The Gateway detects 3 bits of difference (positions 7, 15 and 23) and builds 7 candidates. The initial RW comparison fails with a bit diff of 69. Error Management is triggered and systematically tests all 7 candidates against the Server. Candidates 1 through 6 all fail. Candidate 7 produces an exact match with bit diff = 0 and AUT_OK is returned, demonstrating that the Error Management mechanism correctly exhausts all available candidates before resolving the authentication.

7.8.4. Option 4 – FW Verification (Correct RW)

This test simulates a valid firmware image. The device computes a SHA-256 hash of the firmware, obtains the expected RW from the crypto chip using that hash as CW, and sends FW_OK to the Gateway.

Device terminal:

```
======================================================
  FW VERIFICATION TEST
======================================================

[ FW ] Simulated FW size : 64 bytes
[ FW ] Computing SHA-256 of test FW...
[ FW ] SHA-256:
8ef112fbb6c51ec9fd1a71793f238017d089c73c20eecc24b06f3084b2f693d7
[ FW ] Obtaining expected RW from crypto chip...
[ FW ] Expected RW : 5781e0753e242708e562cda59dd39d03
[ FW ] Using CORRECT RW (expected == computed)

[ FW ] Running SGR_SDK_Verification_FW...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — FW Verification
[MW] ──────────────────────────────────────────
[MW] [1/4] Computing SHA-256 of firmware (64 bytes)...
[MW] [1/4] SHA-256 computed ✓
[MW] HASH: 8ef112fbb6c51ec9fd1a71793f238017d089c73c20eecc24b06f3084b2f693d7
[MW] [2/4] Requesting RW from crypto chip (HASH as CW)...
[MW] [2/4] RW computed by crypto chip ✓
[MW] RW computed: 5781e0753e242708e562cda59dd39d03
[MW] RW expected: 5781e0753e242708e562cda59dd39d03
[MW] [3/4] RW match — firmware integrity OK ✓
[MW] [4/5] Sending FW verification result to GW: FW_OK
[SERVER] FW verification result sent — CRC32: 0x9D98B39D, result: FW_OK, RW:
5781e075...
[MW] [4/5] Result sent ✓ — waiting for cloud ACK...
[SERVER] FW verification ACK received — status: ACK_OK
[MW] [5/5] Cloud ACK received: ACK_OK ✓
------------------------------------------------------
  ✓ FW Verification OK — RW_verify saved for installation
```

Gateway Terminal:

```
[MW] [CH0] ──────────────────────────────────────────
[MW] [CH0] GW — waiting for FW verification result
[MW] [CH0] ──────────────────────────────────────────
[MW] [CH0][1/2] FW verification received — CRC32=0x9D98B39D, result=FW_OK ✓
      [PORT][MQTT] Initializing communication port .....
      [PORT][MQTT] Connected
[MW] [CH0][2/4] Bridging FW verification result to server...
      [PORT][MQTT] TX about to write |
topic=sandgrain/v1/800000000000000000000000000000018b99d9c43a98f7b7021ede082580
9fbf6/req/fwVerification | size=157 |
data={"type":"6","payload":{"TID":"800000000000000000000000000000018b99d9c43a98
f7b7021ede0825809fbf6","FW_VERIFY":"FW_OK","RW":"5781e0753e242708e562cda59dd39
d03"}}
      [PORT][MQTT] Publish OK
[SERVER] FW verification bridged to server — TID: 80000000..., result: FW_OK,
RW: 5781e075...
[MW] [CH0][2/4] FW verification bridged to server ✓
      [PORT][MQTT] RX received (132 bytes) |
data={"type":"6","status":{"code":0,"result":"OK"},"payload":{"permission":"GR
ANTED","verificationRW":"5781e0753e242708e562cda59dd39d03"}}
[SERVER] FW verification ACK received — status: ACK_OK
[MW] [CH0][3/4] ACK received from server: ACK_OK ✓
[MW] [CH0][4/4] ACK forwarded to device ✓
      [PORT][MQTT] Closing communication port ....

[GW] Flow completed OK
```

The server responds with permission: GRANTED and echoes back the verificationRW. The device saves RW_verify locally for use in the subsequent Installation flow.

7.8.5. Option 5 — FW Verification (Wrong RW)

This test simulates a tampered or unsigned firmware image. The application intentionally provides a zeroed RW (00000000...) instead of the value returned by the crypto chip, causing the middleware to detect a mismatch and report FW_NOK.

Device terminal:

```
======================================================
  FW VERIFICATION TEST
======================================================

[ FW ] Simulated FW size : 64 bytes
[ FW ] Computing SHA-256 of test FW...
[ FW ] SHA-256:
8ef112fbb6c51ec9fd1a71793f238017d089c73c20eecc24b06f3084b2f693d7
[ FW ] Obtaining expected RW from crypto chip...
[ FW ] Expected RW : 5781e0753e242708e562cda59dd39d03
[ FW ] Using WRONG RW  (tampered / unsigned FW image)
[ FW ] Wrong RW    : 00000000000000000000000000000000

[ FW ] Running SGR_SDK_Verification_FW...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — FW Verification
[MW] ──────────────────────────────────────────
[MW] [1/4] Computing SHA-256 of firmware (64 bytes)...
[MW] [1/4] SHA-256 computed ✓
[MW] HASH: 8ef112fbb6c51ec9fd1a71793f238017d089c73c20eecc24b06f3084b2f693d7
[MW] [2/4] Requesting RW from crypto chip (HASH as CW)...
```

```
[MW] [2/4] RW computed by crypto chip ✓
[MW] RW computed: 5781e0753e242708e562cda59dd39d03
[MW] RW expected: 00000000000000000000000000000000
[MW] [3/4] RW mismatch — firmware integrity FAILED ✗
[MW] [4/5] Sending FW verification result to GW: FW_NOK
[SERVER] FW verification result sent — CRC32: 0x06A98CD7, result: FW_NOK, RW:
5781e075...
[MW] [4/5] Result sent ✓ — waiting for cloud ACK...
[SERVER] FW verification ACK received — status: ACK_OK
[MW] [5/5] Cloud ACK received: ACK_OK ✓
------------------------------------------------------
  ✗ FW Verification NOK — FW_NOK sent to GW
```

Gateway Terminal:

```
[MW] [CH0] ──────────────────────────────────────────
[MW] [CH0] GW — waiting for FW verification result
[MW] [CH0] ──────────────────────────────────────────
[MW] [CH0][1/2] FW verification received — CRC32=0x06A98CD7, result=FW_NOK ✓
      [PORT][MQTT] Initializing communication port .....
      [PORT][MQTT] Connected
[MW] [CH0][2/4] Bridging FW verification result to server...
      [PORT][MQTT] TX about to write |
topic=sandgrain/v1/80000000000000000000000000000018b99d9c43a98f7b7021ede082580
9fbf6/req/fwVerification | size=158 |
data={"type":"6","payload":{"TID":"80000000000000000000000000000018b99d9c43a98
f7b7021ede0825809fbf6","FW_VERIFY":"FW_NOK","RW":"5781e0753e242708e562cda59dd3
9d03"}}
      [PORT][MQTT] Publish OK
[SERVER] FW verification bridged to server — TID: 80000000..., result: FW_NOK,
RW: 5781e075...
[MW] [CH0][2/4] FW verification bridged to server ✓
      [PORT][MQTT] RX received (132 bytes) |
data={"type":"6","status":{"code":0,"result":"OK"},"payload":{"permission":"DE
NIED","verificationRW":"5781e0753e242708e562cda59dd39d03"}}
[SERVER] FW verification ACK received — status: ACK_OK
[MW] [CH0][3/4] ACK received from server: ACK_OK ✓
[MW] [CH0][4/4] ACK forwarded to device ✓
      [PORT][MQTT] Closing communication port ....

[GW] Flow completed OK
```

Note that even in the FW_NOK case the SDK still forwards the actual crypto chip RW (5781e075...) to the server. The server records the result and responds with permission: DENIED. The device does **not** save RW_verify when the result is FW_NOK.

Important: The Gateway always reports [GW] Flow completed OK for this step because the framing and transport succeeded. The actual rejection of the firmware — and any downstream consequences — is enforced by the Cloud SDK when Installation is subsequently requested.

7.8.6. Option 6 – Installation

7.8.6.1. Installation after FW_OK (Option 4 first)

The device uses the saved RW_verify (from option 4) as the CW for the installation exchange. The crypto chip computes a new RW_install. Both values are forwarded to the server as a type-7 frame.

Device Terminal:

```
======================================================
  INSTALLATION TEST
======================================================

[ INSTALL ] Running SGR_SDK_Installation...
------------------------------------------------------
[MW] ──────────────────────────────────────────
[MW] DEVICE — Installation
[MW] ──────────────────────────────────────────
[MW] [1/4] Using saved RW_verify as CW_install
[MW] CW_install:
5781e0753e242708e562cda59dd39d035781e0753e242708e562cda59dd39d03
[MW] [2/4] Requesting RW_install from crypto chip...
[MW] [2/4] RW_install computed ✓
[MW] RW_install: 7cc8a16a4c15d7cdf368a1d3a41a614a
[MW] [3/4] Sending installation request to GW...
[SERVER] Installation request sent — CRC32: 0x06A98CD7
[MW] [3/4] Request sent ✓ — waiting for installation result...
[SERVER] Installation result received — status: INSTALL_OK
[MW] [4/4] Installation result: INSTALL_OK ✓
------------------------------------------------------
  ✓ Installation OK

======================================================
  INSTALLATION TEST COMPLETE
```

Gateway Terminal:

```
[MW] [CH0] ──────────────────────────────────────
[MW] [CH0] GW — Installation bridge
[MW] [CH0][1/4] Installation request received | CRC32=0x06A98CD7
      [PORT][MQTT] Initializing communication port .....
      [PORT][MQTT] Connected
[MW] [CH0][2/4] Forwarding installation request to Cloud SDK...
      [PORT][MQTT] TX about to write |
topic=sandgrain/v1/800000000000000000000000000018b99d9c43a98f7b7021ede082580
9fbf6/req/fwInstallation | size=209 |
data={"type":"7","payload":{"TID":"800000000000000000000000000018b99d9c43a98
f7b7021ede0825809fbf6","CW":"5781e0753e242708e562cda59dd39d035781e0753e242708e
562cda59dd39d03","RW":"7cc8a16a4c15d7cdf368a1d3a41a614a"}}
      [PORT][MQTT] Publish OK
[MW] [CH0][3/4] Waiting for installation result from Cloud SDK...
      [PORT][MQTT] RX received (...) |
data={"type":"7","status":{"code":0,"result":"OK"},"payload":{"permission":"GR
ANTED",...}}
[MW] [CH0][3/4] Cloud SDK result: INSTALL_OK
[MW] [CH0][4/4] Relaying installation result to device...
      [PORT][MQTT] Closing communication port ....
```

```
[GW] Flow completed OK
```

The CW_install is the RW_verify value doubled (concatenated with itself): 5781e075...5781e075.... This is a protocol-defined construction ensuring that installation can only proceed using the exact RW produced during a successful FW verification.

7.8.6.2. Installation after FW_NOK (Option 5 first)

If option 6 is run after a failed FW verification (option 5), the server rejects the installation request. The Cloud SDK recognises that the previous verification result was FW_NOK for this TID and responds with ErrorResult.FW_INSTALL_NOT_ALLOWED.

Gateway Terminal:

```
[MW] [CH0] ──────────────────────────────────────────
[MW] [CH0] GW — Installation bridge
[MW] [CH0][1/4] Installation request received | CRC32=0x06A98CD7
      [PORT][MQTT] Initializing communication port .....
      [PORT][MQTT] Connected
[MW] [CH0][2/4] Forwarding installation request to Cloud SDK...
      [PORT][MQTT]         TX         about         to         write         |
topic=sandgrain/v1/800000000000000000000000000000018b99d9c43a98f7b7021ede082580
9fbf6/req/fwInstallation               |              size=209              |
data={"type":"7","payload":{"TID":"800000000000000000000000000000018b99d9c43a98
f7b7021ede0825809fbf6","CW":"5781e0753e242708e562cda59dd39d035781e0753e242708e
562cda59dd39d03","RW":"7cc8a16a4c15d7cdf368a1d3a41a614a"}}
      [PORT][MQTT] Publish OK
[MW] [CH0][3/4] Waiting for installation result from Cloud SDK...
[MW] [CH0][3/4] Waiting for installation result from Cloud SDK...
[MW] [CH0][3/4] Waiting for installation result from Cloud SDK...
      [PORT][MQTT]        RX        received        (271        bytes)        |
data={"type":"7","status":{"code":13,"result":"ErrorResult.FW_INSTALL_NOT_ALLO
WED"},"payload":{"tid":"800000000000000000000000000000018b99d9c43a98f7b7021ede0
825809fbf6","verificationRW":"5781e0753e242708e562cda59dd39d03","reason":"fw_v
erification_failed","fw_verify":"FW_NOK"}}
[MW] [CH0][3/4] Cloud SDK result: INSTALL_NOK
[MW] [CH0][4/4] Relaying installation result to device...
      [PORT][MQTT] Closing communication port ....

[GW] Flow completed — AUTH FAILED
```

The server response includes a reason field (fw_verification_failed) and the original fw_verify status (FW_NOK), providing an auditable trail of why the installation was denied. The Gateway propagates the failure back to the device, and the final status is AUTH_FAILED.

7.9. Notes and Limitations

- The PoC is provided as a reference implementation validated on both Linux and Windows (via MinGW/MSYS2).
- The PoC relies on the Portability Layer to access platform services.
  No changes to the Application, Middleware, or Services layers are required to run the PoC.
- The PoC should be considered a reference and validation tool, not a production application.

- Every Device must have its TID registered in the Gateway whitelist before authentication can succeed. If a Device connects but the Gateway rejects it immediately, verify that the device's TID is correctly registered. Unregistered devices are rejected at the first step of the authentication flow with SGR_SDK_NOT_REGISTERED.